

# Displacement Dynamics Specification

D1 Self van Hove, D2 Non-Gaussian Parameter, and D3 Self-Intermediate Scattering

mdstats

2026-07-22

## 1. Purpose and status

This document specifies the D1 self van Hove function, the D2 non-Gaussian parameter, and the D3 self-intermediate scattering function for `mdstats 0.19.84a0`. All three observables are public consumers of the D0 shared displacement engine. They reuse D0 atom selection, coordinate construction, reference-cell handling, drift removal, physical subspace resolution, semantic signatures, and deterministic atom/origin blocking without reconstructing those choices.

The implementations reside in `mdstats.analysis.displacement_dynamics`. The self van Hove function follows L. Van Hove’s space-time correlation framework [1]. The finite histogram support contract, overflow accounting, projected shell measures, direct-moment cross-check, block integration, metadata, and immutable result schema are `mdstats` design.

D1 does not implement the distinct part of the van Hove function. It contains only same-particle displacements and therefore introduces no cross-atom pair terms.

## 2. Physical definition

For measured atoms  $i = 1, \dots, N$  and valid time origins  $t_0$ , let the D0 projected displacement be

$$\Delta \mathbf{s}_i(t; t_0) = B [\mathbf{r}'_i(t_0 + t) - \mathbf{r}'_i(t_0)] \in \mathbb{R}^d,$$

where  $B \in \mathbb{R}^{d \times 3}$  is the resolved orthonormal subspace basis,  $d \in \{1, 2, 3\}$ , and  $\mathbf{r}'$  already includes the selected coordinate and drift conventions.

The self van Hove distribution in that subspace is

$$G_s^{(d)}(\mathbf{s}, t) = \langle \delta^{(d)}(\mathbf{s} - \Delta \mathbf{s}_i(t; t_0)) \rangle_{i, t_0}.$$

D1 stores the isotropically reduced radial histogram in

$$r = \|\Delta \mathbf{s}\| \geq 0.$$

No isotropy assumption is needed to compute the histogram: all samples are binned by radius. Interpreting the resulting scalar density as a function that fully characterizes the displacement field is justified only when directional information is unimportant or the process is isotropic in the selected subspace.

### 3. Public API

```
def compute_self_van_hove(
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    lag_steps: ArrayLike | None = None,
    max_lag: int | None = None,
    origin_stride: int = 1,
    radial_edges: ArrayLike | None = None,
    r_max: float | None = None,
    n_bins: int = 200,
    require_complete_support: bool = False,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    coordinate_mode: CoordinateMode = "laboratory",
    reference_cell: ReferenceCellInput = "initial",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    atom_block_size: int | None = None,
    origin_block_size: int | None = None,
) -> SelfVanHoveResult:
    ...
```

The defaults intentionally use `reference_cell="initial"`, matching the architecture contract for displacement-distribution observables. The default subspace is the full Cartesian three-dimensional space.

### 4. Input contract

#### 4.1 Trajectory and selections

`collection` must be an `AtomisticFrameCollection` with trajectory semantics, at least two frames, continuous fractional coordinates, and a finite, strictly increasing, uniformly sampled physical time axis. D0 performs these checks.

`species` and `atom_indices` are mutually exclusive. An explicit atom-index selection preserves user order. An omitted selection analyzes all atoms.

Drift selection and subtraction follow D0 exactly. A drift selection without a `drift_mode` is invalid. The result signature stores the exact measured and drift-reference atom populations.

#### 4.2 Lag resolution

`lag_steps` and `max_lag` are mutually exclusive.

- Explicit `lag_steps` must be a nonempty, one-dimensional, strictly increasing integer array with unique values in  $[0, T - 1]$ .
- Boolean and floating-point lag arrays are rejected even when numerically integral.
- If `lag_steps` is omitted, D1 reports every saved-frame lag from zero through `max_lag`, inclusive.
- If both are omitted, `max_lag` resolves to  $\lfloor T/2 \rfloor$ , matching the default many-origin MSD window.
- `origin_stride` is a positive integer and selects origins  $0, s, 2s, \dots < T - k$  for lag  $k$ .

For lag  $k$ , the exact sample count is

$$N_k = N \left( \left\lfloor \frac{T-1-k}{s} \right\rfloor + 1 \right).$$

Lag zero is valid and must produce exact zero direct second moment.

### 4.3 Analysis subspace

`axes` and `projection_basis` are mutually exclusive and are resolved through `AnalysisSubspace`.

- `axes` is a nonempty, unique sequence drawn from "x", "y", and "z";
- axis order is retained in the D0 sample coordinates;
- a general projection basis has shape  $(d, 3)$  and orthonormal rows; and
- the radial norm is evaluated only after projection.

Changing the subspace changes both the displacement radius and the shell measure. A dimensional divisor cannot reinterpret a previously computed three-dimensional histogram.

## 5. Radial support contract

### 5.1 Explicit edges

`radial_edges` and `r_max` are mutually exclusive. Explicit edges must be:

- one-dimensional with at least two values;
- finite and strictly increasing;
- exactly anchored at zero; and
- positive at the final edge.

Bins are left-closed and right-open,  $[r_j, r_{j+1})$ , except that the final bin includes its right endpoint. Therefore a sample exactly equal to `radial_edges[-1]` belongs to the final bin. Only samples strictly greater than the final edge overflow.

When explicit edges are supplied, `n_bins` is validated but does not modify the edge array. This behavior is recorded in metadata.

### 5.2 Generated support

When explicit edges are absent, `n_bins` must be a positive integer.

- With user-supplied `r_max`, D1 constructs `n_bins` uniform bins from zero through the finite positive endpoint.
- With neither support input, D1 performs a deterministic first pass over the same D0 displacement blocks, finds the largest observed projected radius, and constructs uniform complete support.
- For a nonzero observed maximum, the generated endpoint is one `float64` representable value above that maximum. This ensures that a second deterministic pass cannot classify the maximum as overflow because of an endpoint-rounding difference.
- If every observed displacement is exactly zero, D1 uses the positive numerical support

$$r_{\text{auto}} = \sqrt{\epsilon_{64}} \max(1 \text{ \AA}, \max |r'_{i\alpha}(t)|).$$

This avoids a degenerate zero-width histogram. It is a numerical binning scale only; the separately accumulated direct second moment remains exactly zero.

The automatic prepass uses the same lags, origin stride, subspace, and resolved D0 block plan as the accumulation pass.

### 5.3 Overflow and strict support

Samples outside finite user support are never silently discarded and the in-range histogram is never renormalized to one. For lag  $k$ ,

$$P_{kj} = \frac{C_{kj}}{N_k}, \quad P_k^{\text{overflow}} = \frac{C_k^{\text{overflow}}}{N_k},$$

and the exact counting identity is

$$\sum_j C_{kj} + C_k^{\text{overflow}} = N_k.$$

Thus

$$\sum_j P_{kj} + P_k^{\text{overflow}} = 1$$

up to floating-point division roundoff. `require_complete_support=True` raises a `ValueError` after accumulation if any overflow count is nonzero. The result is not returned in that case.

## 6. Shell measure and density

For edges  $r_j < r_{j+1}$ , the full-space shell measure in the selected subspace is

$$\mu_j^{(1)} = 2(r_{j+1} - r_j),$$

$$\mu_j^{(2)} = \pi(r_{j+1}^2 - r_j^2),$$

$$\mu_j^{(3)} = \frac{4\pi}{3}(r_{j+1}^3 - r_j^3).$$

The one-dimensional radius is  $|s|$ . The factor of two accounts for the two signed directions represented by one nonnegative radial bin.

D1 stores

$$G_{s,kj} = \frac{P_{kj}}{\mu_j^{(d)}}.$$

Therefore

$$\sum_j G_{s,kj} \mu_j^{(d)} = 1 - P_k^{\text{overflow}}.$$

The density has units  $\text{\AA}^{-d}$ . `shell_probability` is dimensionless, and `shell_measure` has units  $\text{\AA}^d$ .

Radial centers are arithmetic bin midpoints and are supplied only for plotting or approximate quadrature. They are not used to define the density or direct moments.

## 7. Direct second moment

D1 accumulates the unbinned projected second moment over every sample, including samples beyond finite histogram support:

$$M_2(k) = \frac{1}{N_k} \sum_{i, t_0} \|\Delta \mathbf{s}_i(k; t_0)\|^2.$$

For the full Cartesian subspace and identical input semantics, this quantity must agree with the D0 direct many-origin MSD. For a projected subspace, it must agree with direct projection of the corresponding displacement second-moment tensor.

A center-based histogram estimate,

$$\sum_j r_{j, \text{center}}^2 P_{kj},$$

is only approximate and excludes overflow. D1 does not substitute it for the direct moment.

## 8. Result schema

```
@dataclass(frozen=True, slots=True)
class SelfVanHoveResult:
    lag_steps: NDArray[np.int64]           # (L,)
    lag_times: NDArray[np.float64]         # (L,), ps
    radial_edges: NDArray[np.float64]       # (B + 1,), Angstrom
    radial_centers: NDArray[np.float64]     # (B,), Angstrom
    shell_measure: NDArray[np.float64]      # (B,), Angstrom**d
    shell_probability: NDArray[np.float64]  # (L, B)
    density: NDArray[np.float64]           # (L, B), Angstrom**(-d)
    counts: NDArray[np.int64]              # (L, B)
    overflow_counts: NDArray[np.int64]      # (L,)
    overflow_probability: NDArray[np.float64] # (L,)
    n_samples: NDArray[np.int64]           # (L,)
    direct_second_moment: NDArray[np.float64] # (L,), Angstrom**2
    atom_indices: NDArray[np.int64]        # (N,)
    projection_basis: NDArray[np.float64]   # (d, 3)
    signature: DynamicsInputSignature
    metadata: Mapping[str, Any]
```

The result also exposes the derived read-only property `captured_probability = 1.0 - overflow_probability`.

All arrays are owned, C-contiguous, and read-only. Metadata is recursively immutable. Constructor validation enforces:

- all documented shapes and finite floating-point values;
- strictly increasing lags and radial edges;
- exact radial-center and shell-measure identities;
- nonnegative integer counts and positive sample counts;
- exact count conservation;
- probability/count and density/shell-probability identities;
- nonnegative direct second moments within strict roundoff tolerance;
- atom and projection agreement with the signature; and
- valid three-dimensional source coordinates represented by the signature.

## 9. Algorithm

1. Validate booleans, positive integer controls, lag-choice exclusivity, and support-choice exclusivity.
2. Call `prepare_displacement_inputs` exactly once with the requested atom, coordinate, drift, and subspace semantics.
3. Resolve the lag array and one D0 `DisplacementBlockPlan` using the package displacement-memory target.
4. Validate explicit support, generate support from `r_max`, or make the deterministic automatic maximum-radius prepass.
5. Construct radial centers and dimension-correct shell measures.
6. Iterate D0 blocks in deterministic lag/origin/atom order.
7. Compute radii with `numpy.linalg.norm` along the projected component axis.
8. Accumulate direct squared-radius sums before applying histogram support.
9. Count values greater than the final edge as overflow. Histogram all remaining values using the exact endpoint policy.
10. Divide by the total sample count, not the captured count.
11. Divide shell probabilities by shell measures to form the density.
12. Enforce strict-support policy, construct the immutable result, and validate all invariants.

No complete displacement tensor, radius tensor, or per-sample history is materialized. Peak displacement workspace remains bounded by the D0 plan; D1 adds one block-sized radius array and fixed-size  $O(LB)$  accumulators.

## 10. Metadata

At minimum, metadata records:

- estimator name and D1 contract version;
- radial-support mode: `"explicit_edges"`, `"user_r_max"`, or `"automatic_complete"`;
- whether `n_bins` controlled the support;
- bin count and final support radius;
- bin endpoint convention;
- shell dimension and density units;
- origin stride;
- resolved atom and origin block sizes;
- D0 bytes per sample, estimated peak displacement work, and memory target;
- total overflow count and whether support was required complete;
- coordinate, reference-cell, drift, and projection summaries inherited from the prepared bundle; and
- the complete `DynamicsInputSignature` as a dedicated result field.

## 11. Edge cases and failure rules

- An ensemble, missing time axis, nonuniform time grid, or fewer than two frames is rejected by D0.
- Empty atom selections are rejected.
- `lag_steps` combined with `max_lag` is rejected.
- Negative, out-of-range, repeated, unsorted, floating, or boolean lags are rejected.
- Boolean values are rejected for integer controls.
- `radial_edges` combined with `r_max` is rejected.
- Edges not starting at zero, repeated edges, decreasing edges, nonfinite edges, or a nonpositive final edge are rejected.
- Nonpositive or nonfinite `r_max` is rejected.
- `require_complete_support` must be an actual boolean.
- A rank-deficient or nonorthonormal projection basis is rejected.

- Zero-displacement data use a finite automatic numerical support and remain exactly zero in `direct_second_moment`.
- A sample exactly on the final edge is captured. A sample one representable value above it overflows.
- Strict support raises if any lag overflows, including when only one sample is outside support.

## 12. Required focused tests

### 12.1 Deterministic analytic cases

- static atoms, including automatic zero-displacement support;
- uniform translation with exact lag-dependent radii;
- explicit atom order and Cartesian-axis order;
- a genuinely rotated one-dimensional projection;
- exact final-edge ownership and one-value-above-edge overflow;
- strict-support rejection.

### 12.2 Normalization and moments

- one-, two-, and three-dimensional shell measures;
- exact count conservation at every lag;
- density integrated against shell measure equals captured probability;
- automatic support has zero overflow;
- direct second moment agrees with D0/MSD for full Cartesian samples;
- projected direct moment agrees with a hand calculation;
- center-based histogram moment converges toward the direct moment as bin width decreases.

### 12.3 Statistical cases

- deterministic seeded Gaussian increments in one, two, and three dimensions;
- expected second moment  $d\sigma^2$  within finite-sample tolerance;
- radial density/probability remains normalized with the appropriate shell measure.

### 12.4 Blocking, provenance, and immutability

- atom-block and origin-block invariance;
- deterministic repeatability;
- signature preserves trajectory, measured atoms, drift atoms, coordinate mode, reference cell, and projection;
- every public array rejects mutation;
- nested metadata rejects mutation; and
- public exports resolve from both `mdstats` and `mdstats.analysis`.

### 12.5 Invalid inputs

- all lag, support, selector, projection, boolean, and block-size failures listed in Section 11.

## 13. D2 physical definition

For the same projected D0 displacement samples used by D1, define

$$m_2(t) = \langle r^2(t) \rangle, \quad m_4(t) = \langle r^4(t) \rangle,$$

where

$$r(t) = \|B\Delta\mathbf{r}(t)\|$$

and  $d = \text{rank } B \in \{1, 2, 3\}$ . The dimension-correct non-Gaussian parameter is

$$\alpha_2^{(d)}(t) = \frac{d}{d+2} \frac{m_4(t)}{m_2(t)^2} - 1.$$

This displacement-cumulant diagnostic follows the liquid-scattering treatment of Rahman, Singwi, and Sjolander [2]. Applying the same formula after changing only a scalar dimensional divisor is forbidden:  $r$  and  $d$  must be resolved from the same physical subspace. The D0 projection contract supplies both.

For an isotropic Gaussian displacement distribution in the resolved  $d$  dimensions,  $\alpha_2^{(d)} = 0$ . A distribution concentrated on one fixed radius has

$$\alpha_2^{(d)} = -\frac{2}{d+2}.$$

Positive values indicate broader tails or heterogeneous displacement scales relative to a Gaussian with the same second moment. D2 is a diagnostic of the radial displacement distribution; it does not by itself identify a unique microscopic mechanism.

## 14. D2 public API

```
def compute_non_gaussian_parameter(
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    max_lag: int | None = None,
    origin_stride: int = 1,
    lag_stride: int = 1,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    coordinate_mode: CoordinateMode = "laboratory",
    reference_cell: ReferenceCellInput = "initial",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    atom_block_size: int | None = None,
    origin_block_size: int | None = None,
) -> NonGaussianResult:
    ...
```

The defaults intentionally match D1 displacement semantics: full Cartesian space, laboratory coordinates, the initial reference-cell convention when a reference mapping is requested, all atoms, no drift removal, and a half-record many-origin lag window.

## 15. D2 input and lag contract

Trajectory, measured-selection, drift-selection, coordinate, reference-cell, and subspace validation are delegated to D0 exactly as in D1.

- `max_lag` is an inclusive saved-frame lag and defaults to  $\lfloor T/2 \rfloor$ ;



- `origin_stride` and `lag_stride` are strict positive integers and reject booleans;
- reported lags are  $0, s_\ell, 2s_\ell, \dots \leq k_{\max}$ , where  $s_\ell$  is `lag_stride`;
- the exact sample count at lag  $k$  is

$$N_k = N \left( \left\lfloor \frac{T-1-k}{s_o} \right\rfloor + 1 \right),$$

where  $s_o$  is `origin_stride`; and

- at least one lag is always returned, including lag zero.

`axes` and `projection_basis` are mutually exclusive. The resolved basis must have orthonormal rows. Its rank is the  $d$  used in the prefactor, and norms are computed only after projection.

## 16. D2 zero-moment and numerical contract

`alpha2` is undefined whenever the directly accumulated second moment is exactly zero. This rule applies at every stored lag, not only lag zero. D2 stores

- `alpha2 = NaN`; and
- `undefined_mask = True`

at each such lag. Because every sample contribution is nonnegative, an exact zero second moment requires an exact zero fourth moment; the result constructor validates this invariant.

At a defined lag, `alpha2` must be finite. D2 accumulates unbinned moments from all samples and never estimates moments from D1 histogram centers. Intermediate second- or fourth-power overflow is rejected with `ValueError`; silent infinities, clipping, or finite-value substitution are forbidden.

Atom and origin blocking may alter floating-point reduction grouping. Results from different valid block plans must agree to a strict numerical tolerance, not necessarily bit for bit.

## 17. D2 result schema

```
@dataclass(frozen=True, slots=True)
class NonGaussianResult:
    lag_steps: NDArray[np.int64]
    lag_times: NDArray[np.float64]
    second_moment: NDArray[np.float64]
    fourth_moment: NDArray[np.float64]
    alpha2: NDArray[np.float64]
    undefined_mask: NDArray[np.bool_]
    n_samples: NDArray[np.int64]
    atom_indices: NDArray[np.int64]
    projection_basis: NDArray[np.float64]
    signature: DynamicsInputSignature
    metadata: Mapping[str, Any]
```

All arrays are owned and read-only. Metadata is recursively immutable. The constructor validates shapes, finite nonnegative moments, exact undefined-mask semantics, signature consistency, lag-time consistency, and recomputed  $\alpha_2$  values.

Required metadata includes:

- estimator and contract version;
- borrowed-theory citation and DOI;

- subspace rank and dimensionless `alpha2` units;
- second- and fourth-moment units;
- origin and lag strides;
- resolved atom/origin block sizes and memory target;
- undefined lag indices and count; and
- the complete immutable D0 input metadata.

## 18. D2 algorithm

1. Validate strict integer controls and prepare one D0 displacement bundle.
2. Resolve the inclusive maximum lag and regular lag sequence.
3. Resolve one D0 atom/origin block plan under the standard memory target.
4. Iterate blocks in deterministic D0 order.
5. Compute stable projected radii, then accumulate  $r^2$ ,  $r^4$ , and exact sample counts per lag.
6. Verify counts against the analytic origin-count formula.
7. Divide accumulated sums by total samples to obtain  $m_2$  and  $m_4$ .
8. Mark every exact-zero  $m_2$  lag undefined and evaluate  $\alpha_2$  only on the remaining lags.
9. Construct the immutable result with the D0 signature and audit metadata.

D2 performs no histogram allocation and no second displacement prepass. It is a single-pass moment observable over the prepared displacement samples.

## 19. D2 required focused tests

### 19.1 Analytic distributions

- static trajectories: all moments zero and all reported lags undefined;
- deterministic translation or fixed-radius shell:  $\alpha_2 = -2/(d + 2)$  for nonzero lags;
- seeded isotropic Gaussian increments in ranks one, two, and three:  $\alpha_2 \approx 0$  within statistical tolerance; and
- a two-population displacement-scale mixture: positive  $\alpha_2$ .

### 19.2 Cross-observable consistency

- D2 second moments agree with direct D0/MSD on identical regular lags;
- D2 second moments agree with D1 `direct_second_moment`;
- projected and rotated-basis cases use the same rank in the norm and prefactor;
- later-lag exact zero moments remain undefined even when earlier lags are defined.

### 19.3 Engineering contract

- atom/origin block invariance;
- explicit atom-order preservation;
- exact sample-count conservation;
- signature and nested metadata preservation;
- deep immutability;
- root and analysis exports;
- constructor rejection of inconsistent masks, moments, or signatures; and
- strict rejection of malformed selectors, projections, lags, booleans, and block controls.

## 20. D3 physical definitions

D3 implements the self-intermediate scattering function, the spatial Fourier transform of the self part of the van Hove correlation. The underlying self-correlation formalism follows Van Hove [1] and Vineyard [3]. The

two public modes intentionally preserve different information.

## 20.1 Explicit-vector mode

For an admissible laboratory reciprocal vector  $\mathbf{q} \in \mathbb{R}^3$ , D3 computes

$$F_s(\mathbf{q}, t) = \langle \exp(i\mathbf{q} \cdot \Delta \mathbf{r}_i(t; t_0)) \rangle_{i, t_0}.$$

Let  $B \in \mathbb{R}^{d \times 3}$  be the D0 orthonormal row basis. The selected subspace projector is  $P = B^\top B$ . An explicit vector is admissible only when

$$\|\mathbf{q} - \mathbf{q}P\|_2 \leq \tau_q \max(1, \|\mathbf{q}\|_2),$$

with  $\tau_q = 10^{-10}$ . Admissible projected coordinates are  $\mathbf{q}_d = \mathbf{q}B^\top$ , and the phase is evaluated as  $\mathbf{q}_d \cdot \Delta \mathbf{s}$ . Components outside the selected subspace are rejected; they are never discarded silently.

Explicit-vector values are complex. Input vector order and duplicates are preserved exactly. The zero vector gives exactly one at every lag. If both  $\mathbf{q}$  and  $-\mathbf{q}$  are requested, their results obey complex conjugation up to floating-point reduction error.

## 20.2 Isotropic-magnitude mode

For a nonnegative magnitude  $q$ , D3 angularly averages the directional phase in the resolved  $d$ -dimensional subspace. With  $r = \|\Delta \mathbf{s}\|$ , the kernels are

$$K_1(qr) = \cos(qr), \quad K_2(qr) = J_0(qr), \quad K_3(qr) = j_0(qr) = \frac{\sin(qr)}{qr}.$$

Therefore

$$F_s^{\text{iso}, d}(q, t) = \langle K_d(qr) \rangle.$$

$J_0$  and the spherical Bessel function  $j_0$  are evaluated with SciPy's special-function implementation [4]. D3 overwrites all  $q = 0$  columns with exactly one after accumulation. Isotropic values are real even when the underlying displacement distribution is anisotropic; they are the angular average of the characteristic function, not a claim that the trajectory is isotropic.

## 21. D3 public API

```
def compute_self_intermediate_scattering(
    collection: AtomisticFrameCollection,
    *,
    q_vectors: ArrayLike | None = None,
    q_magnitudes: ArrayLike | None = None,
    isotropic: bool = True,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    max_lag: int | None = None,
    origin_stride: int = 1,
    lag_stride: int = 1,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
```

```

coordinate_mode: CoordinateMode = "laboratory",
reference_cell: ReferenceCellInput = "initial",
drift_mode: DriftMode | None = None,
drift_species: SpeciesSelection = None,
drift_atom_indices: ArrayLike | None = None,
atom_block_size: int | None = None,
origin_block_size: int | None = None,
) -> SelfIntermediateScatteringResult:
    ...

```

`isotropic` is a strict Boolean. Exactly one `q` representation is accepted:

- `isotropic=True` requires `q_magnitudes` and rejects `q_vectors`;
- `isotropic=False` requires `q_vectors` and rejects `q_magnitudes`.

No implicit mode inference is performed.

## 22. D3 q-space contract

### 22.1 Magnitudes

`q_magnitudes` must be a nonempty one-dimensional real array containing only finite nonnegative values in inverse angstrom. Boolean arrays are rejected. Input ordering and duplicate values are retained. Negative zero is normalized to zero in the owned result array.

### 22.2 Vectors

`q_vectors` must have shape  $(Q, 3)$  with  $Q \geq 1$ , contain finite real values, and use inverse-angstrom units. Boolean arrays are rejected. Every vector is checked against the selected D0 projector using the tolerance in Section 20.1. The result stores both the original three-dimensional vectors and their  $d$ -component projected coordinates. Ordering and duplicates are retained.

D3 does not interpret vectors as reciprocal-lattice indices and does not apply  $2\pi$  factors. Callers supply Cartesian wavevectors directly in  $\text{\AA}^{-1}$ .

## 23. D3 lag, sample, and blocking contract

D3 uses the D2 regular-lag convention:

- `max_lag` defaults to  $\lfloor T/2 \rfloor$ ;
- `lag_stride` is a positive integer;
- reported lags are 0, `lag_stride`, ...,  $\leq \text{max\_lag}$ ;
- `origin_stride` selects valid origins independently; and
- every lag has the exact D0 sample count

$$N_k = N \left( \left\lfloor \frac{T-1-k}{s} \right\rfloor + 1 \right).$$

D3 consumes the same deterministic lag-major, origin-block-major, atom-block-major iterator as D1-D2. It additionally evaluates `q` values in private contiguous chunks under a fixed transient-work target. `Q` chunking may change floating-point reduction grouping but cannot change `q` order, `q` multiplicity, sample counts, or physical semantics. The output allocation itself is not hidden by chunking and has shape  $(L, Q)$ .

## 24. D3 result schema

```
@dataclass(frozen=True, slots=True)
class SelfIntermediateScatteringResult:
    lag_steps: NDArray[np.int64]
    lag_times: NDArray[np.float64]
    values: NDArray[np.float64] | NDArray[np.complex128]
    q_magnitudes: NDArray[np.float64] | None
    q_vectors: NDArray[np.float64] | None
    projected_q_vectors: NDArray[np.float64] | None
    n_samples: NDArray[np.int64]
    atom_indices: NDArray[np.int64]
    projection_basis: NDArray[np.float64]
    signature: DynamicsInputSignature
    metadata: Mapping[str, Any]
```

Exactly one  $q$  representation is populated. In isotropic mode, `values` has `float64` dtype and `projected_q_vectors` is `None`. In explicit-vector mode, `values` has `complex128` dtype and `projected_q_vectors` has shape  $(Q, d)$ . All arrays are owned and read-only, and metadata is recursively immutable.

The constructor validates:

- lag ordering, shapes, finite  $q$  inputs, and positive sample counts;
- mode-consistent value dtype and  $q$  representation;
- finite real and imaginary values;
- orthonormal projection basis and signature consistency;
- reconstruction of projected  $q$  coordinates;
- rejection of out-of-subspace vector components;
- exact unit values at lag zero and at every zero  $q$ ; and
- lag times against the signature sample spacing.

Required metadata includes estimator and contract version, mode, borrowed references,  $q$  and time units, subspace rank,  $q$  count,  $q$ -space tolerance, origin/lag strides, D0 block-plan details, private  $q$ -chunk details, output byte count, and complete immutable D0 input metadata.

## 25. D3 numerical algorithm

1. Strictly validate the mode and its one allowed  $q$  representation.
2. Prepare one D0 displacement bundle and resolve regular lags and one block plan.
3. Validate vectors against the D0 projector, or validate magnitudes.
4. Allocate  $(L, Q)$  real or complex extended-precision accumulators and exact integer sample counts.
5. For every D0 block, flatten its projected displacement samples without changing sample membership.
6. Process contiguous  $q$  chunks:
  - isotropic mode computes stable radii once and accumulates  $K_d(qr)$ ;
  - vector mode computes projected phases and accumulates  $\cos \phi + i \sin \phi$ .
7. Reject non-finite phase arguments, kernel values, or accumulated values.
8. Verify sample counts against the analytic D0 formula and divide by counts.
9. Set zero-lag and zero- $q$  values exactly to one.
10. Cast to `float64` or `complex128` and construct the immutable result.

D3 never derives scattering values from D1 histogram centers. A D1 transform is only a discretized validation comparison. Direct D0 samples are the scientific estimator.

## 26. D3 required focused tests

### 26.1 Exact identities

- lag zero equals one for every  $\mathbf{q}$  in both modes;
- zero magnitude and zero vector equal one at every lag;
- deterministic ballistic motion matches  $\exp(i\mathbf{q} \cdot \mathbf{v}t)$  in vector mode;
- requested  $\pm\mathbf{q}$  pairs are complex conjugates; and
- one-dimensional isotropic mode equals the mean cosine for the same samples.

### 26.2 Statistical and cross-observable validation

- seeded Gaussian increments agree with  $\exp(-q^2\sigma_t^2/2)$  in ranks one, two, and three;
- two-dimensional and three-dimensional kernels agree with direct SciPy  $J_0$  and  $j_0$  sample averages;
- a fine complete D1 radial histogram numerically transforms to D3 isotropic values within histogram discretization error; and
- rotated subspaces use the same projected displacement and vector coordinates.

### 26.3 Engineering contract

- isotropic/vector mutual exclusion and strict Boolean validation;
- malformed, negative, non-finite, or empty  $\mathbf{q}$  inputs rejected;
- out-of-subspace vector components rejected;
- exact  $\mathbf{q}$  ordering and duplicates retained;
- atom/origin block invariance and exact sample counts;
- explicit atom-order preservation;
- constructor invariant rejection;
- deep immutability and signature preservation;
- root and analysis exports; and
- unchanged D0-D2, MSD, VACF, transport, spectrum, and plotting regressions.

## 27. Acceptance condition

D3 is complete only when:

1. this Markdown specification is finalized before implementation;
2. the corresponding PDF compiles and passes visual inspection;
3. source comments and docstrings cite Van Hove [1], Vineyard [3], and SciPy [4] where their definitions or special functions are used;
4. `SelfIntermediateScatteringResult` and `compute_self_intermediate_scattering` consume D0 without reconstructing displacement preparation;
5. focused D0-D3, MSD, VACF, transport, spectrum, and plotting tests pass;
6. public exports, changelog, README, release records, and architecture manual agree on 0.19.84a0; and
7. compilation, import smoke, PDF preflight, archive import, checksum, and ZIP integrity gates pass.

## References

- [1] L. Van Hove, “Correlations in Space and Time and Born Approximation Scattering in Systems of Interacting Particles,” *Physical Review* **95**, 249-262 (1954). DOI: 10.1103/PhysRev.95.249.
- [2] A. Rahman, K. S. Singwi, and A. Sjolander, “Theory of Slow Neutron Scattering by Liquids. I,” *Physical Review* **126**, 986-996 (1962). DOI: 10.1103/PhysRev.126.986.

[3] G. H. Vineyard, “Scattering of Slow Neutrons by a Liquid,” *Physical Review* **110**, 999-1010 (1958). DOI: 10.1103/PhysRev.110.999.

[4] P. Virtanen, R. Gommers, T. E. Oliphant, et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods* **17**, 261-272 (2020). DOI: 10.1038/s41592-019-0686-2.